Equivariant GCN for Molecule Dipole Moment Prediction

Ksenia Krasikova[1*†], **Payal Priyadarshini**[1*†], **Philipp Harlos**[1*†]

[1]Department of Mathematics and Informatics, FU Berlin, Berlin, DE

***For correspondence:**
ksek00@zedat.fu-berlin.de (KK);
payap02@zedat.fu-berlin.de (PP);
harlop97@zedat.fu-berlin.de (QQ)

[†]These authors contributed equally to this work

Abstract

Goal of the project: To implement an equivariant graph convolutional network that incorporates 3D atom positions and is invariant/equivariant under rigid body motions (rotation and translation) for predicting the electric dipole moment from the QM9 dataset.
Main results: The model yields better performance than the base GCN model and has performance comparable to that of the hyperbolic GCN from previous project.
Personal key learnings: Translation of the base GCN model to equivariant GCN and methods of debuging a faulty (NaN loss) training process.
Estimated working hours: 10
Project evaluation: 4
Number of Words: 1161

Goal

This week's project centers on the implementation of a equivariant Graph Convolutional Network (GCN), building upon concepts discussed inthe lecture (***von Tycowicz, 2025***). Using a pre-designed code tempate and the QM9 benchmarking dataset, the primary goal is to develop and train a GCN model more specifically for predicting the electric dipole moment of small organic molecules. In this framework, each molecule is modeled as a graph where an individual atom functions as nodes and the chemcical bonds between two atoms represent the edges.

Data

For this project, we utilize the QM9 dataset, which is a widely used benchmark in machine learning for molecular properties. It comprises data for over 130,000 small organic molecules, each of which are annotated with various chemical properties. We are specifically focusing on predicting the electric dipole moment.

Pre-processing

First, as in previous projects, all prediction targets except from dipole moment are dropped/. Then, the data is split into training, test and validation data, each with 1000 molecules.
The initially loaded data is using PyTorch Geometric library (***geometric.data.Data***) . This is then converted into the **jraph.GraphsTuple** format, to ensure data compatability with the sunsequent workflow. For the transformation, each attribute like atom features, edge connectivity, and the chosen target property, along with the 3D atomic positions, is manually assigned into a structured unified object known as **GraphsTuple**. The number of nodes and edges are stored in the **GraphsTuple.n_node** and **GraphsTuple.n_edge** respectively.
Lastly, the batching groups individual graph data points into batches with the help of custom function. As there are varying numbers of atoms and chemical bonds per graph, graphs are first "con-

catinated" together and in the end extra padding is added. This further ensures that all the graphs that are within the batch conform to predefined maximum dimensions.

Methods

Layers and Model

The first component of our Equivariant Graph Convolutional Network is a stack of Equivariant Graph Convolutional Layers (EGCLs), each designed to update both the atomic features and the 3D positions of the nodes in a molecular graph. Compared to the previous week, each node's state is represented not only by its categorical attributes (x) but also by its 3D coordinates (p), which are combined in the Feature class for each node.

Each Equivariant Graph Convolution layer consists of two neural networks, psi and phi, as in the standard GCN. Edge messages are computed as before, based on the features of the sender and receiver nodes. However, in addition, we now also use the squared distance between sender and receiver nodes (which is computationally more efficient than using the Euclidean distance). The psi network computes both the messages and the position updates. The phi network then aggregates these messages and updates the node features, just as before. After the update, we apply layer normalization to the new node features to help stabilize training and improve convergence. In earlier experiments without normalization, we often observed the loss reaching huge values, sometimes in the millions, or even resulting in NaNs during training.

After stacking several of these equivariant layers, we additionally include an Invariant Graph Convolutional Layer (IGCL). This layer only updates the node features (x) but leaves the positions (p) unchanged. It also utilizes both categorical features and geometric distances, but its update is designed to be strictly invariant to any rotation or translation of the molecule. In other words, the invariant layer processes the now geometry-aware node features. Further, it refines them for the final prediction, while ensuring that all outputs are independent of the molecule's orientation in space. After the invariant layer, a mean pooling operation condenses the node features of each molecule into a single graph-level embedding. This ensures that, regardless of the number of nodes, we have a fixed-size array representation for every graph. This pooled representation is finally passed through a dense layer to predict the scalar molecular property (the electric dipole moment).

These final layers ensure that the model's predictions are invariant to both node order and the orientation or position of the molecule in space.

Equivariance and Invariance

Just as in the previous assignment, we verified that our model respects the key symmetries necessary for graph learning. As before, we tested layer-level permutation equivariance by shuffling the node order and updating the edge indices, the model's internal outputs changed accordingly, confirming the equirariant behavior.

What is new this week is that our model also needs to handle the geometric symmetries of 3D space. To test this, we applied random rotations and translations to the atomic coordinates. For this, the updated node features remain unchanged (invariant), while the updated positions changed in precisely the same way as the input, showing the desired equivariance. Finally, we also verified that the final model prediction remains precisely the same, regardless of how the molecule is rotated, shifted, or how its atoms are arranged.

Training and Evaluation

This part of the assignment was identical to last week. The model was trained on the QM9 dataset, as in previous assignments, using a 1:1:1 split of 1,000 molecules each for training, validation, and testing. We stacked four Equivariant Graph Convolutional Layers, each with 128 hidden units for psi and phi. As before, we used the Adam optimizer with a learning rate of 1×10^{-4}.

For evaluation, we again compared three loss functions: mean squared error (MSE), mean absolute error (MAE), and Huber loss, training a separate model for each one. We trained for 20 epochs, and after each epoch, we recorded both the training and validation losses to track performance. After training, we reported the test loss for each loss function to evaluate generalization and plotted the training and validation losses over the 20 epochs.

Results & Discussion

First, to ensure the correctness of the invariance and equivariance assumptions, we evaluated unit tests for both individual layers and the overall model. All tests confirmed the intended invariance or equivariance properties, indicating that the implementation is consistent with these assumptions.

Following training, we analyzed the training process by examining the training and validation loss curves for all three loss functions, as shown in ***Figure A1***, ***Figure A2***, and ***Figure A3***. In each case, both the training and validation losses demonstrate a general downward trend. Notably, the validation loss exhibits greater fluctuation over the epochs, whereas the training loss typically follows a smoother descent. An exception is observed in the MSE training loss, which also displays increased variability during training.

On the test set, the equivariant GCN achieves an MSE loss of 1.29 D^2, an MAE loss of 0.76 D, and a Huber loss of 0.49 D. Compared to both the standard (basic) and hyperbolic GCN implementations, the equivariant GCN outperforms the base model, with improvements of approximately 3 D^2 in MSE, 1.6 D in MAE, and 0.7 D in Huber loss. Relative to the hyperbolic GCN, the equivariant GCN achieves comparable performance across all loss metrics.

These results indicate that incorporating equivariance into the GCN architecture provides a performance advantage over the standard GCN, particularly in capturing relevant structural properties of the data. Moreover, the similarity in results between the equivariant and hyperbolic GCNs suggests that both approaches are well-suited for this prediction task.

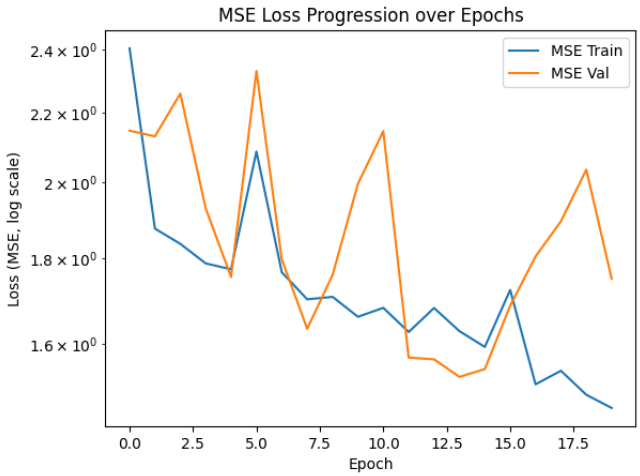
Contributions

1. Code implementation (Layer Functions, code corrections), writing Results and Discussion section by Ksenia Krasikova.
2. Abstract, goal, data, preprocessing, and attempts on Invariant/Equivariant layer implementation by Payal Priyadarshini.
3. Worked on code implementation (Unit tests, GCN Model and Training), writing Method Section by Philipp Harlos.

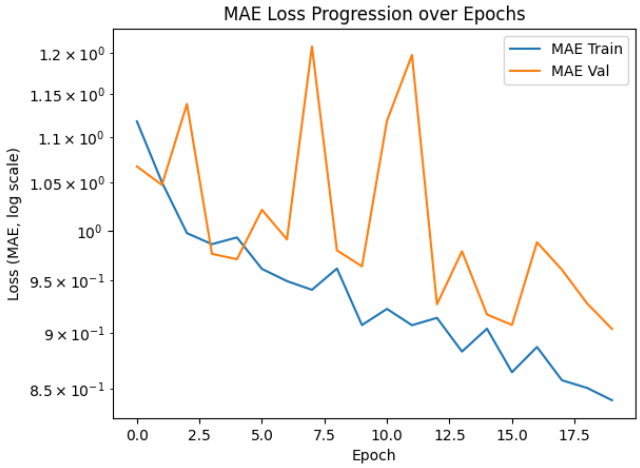
References

von Tycowicz C, Equivariant Graph Neural Networks; 2025. Lecture in "Machine Learning and AI in the Life Sciences: Methods and Applications", Summer Term 2025, July 18, 2025.

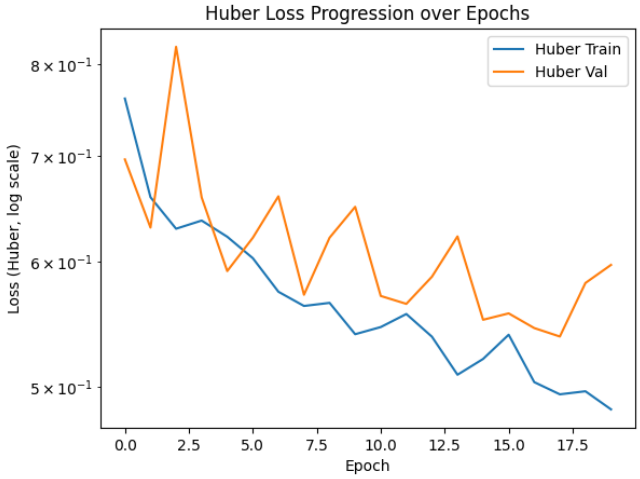
Supplementary Figures



Appendix 0—figure A1. Training and validation loss curves (MSE, log scale) for the GCN model over 20 epochs.



Appendix 0—figure A2. Training and validation loss curves (MAE, log scale) for the GCN model over 20 epochs.



Appendix 0—figure A3. Training and validation loss curves (Huber Loss, log scale) for the GCN model over 20 epochs.